



DALHOUSIE
UNIVERSITY

CSCI 5308 - Advance Topic in Software Development

Adaptive Goal Planner

Final Deliverables Report

GROUP 02

Vijay Puttarevaiah - B01036159

Manavraj Karansinh Thakor - B01075422

Hinesh Jayeshkumar Patel - B01028997

Aditya Natvarbhai Lad - B01061521

Table of Contents

1. Team Members.....	3
2. Project Summary.....	3
3. Sprint Overview.....	4
4. User Stories - Detailed Breakdown.....	4
4.1 Story 1: System Setup and Requirement Gathering.....	4
4.2 Story 2: System Architecture.....	5
4.3 Story 3: User Accounts and Secure Access.....	5
4.4 Story 4: Goal Category Detection and Model Selection.....	6
4.5 Story 5: Goal Creation and Timeline Definition.....	7
4.6 Story 6: Progress Tracking and Task Completion.....	8
4.7 Story 7: Adaptive Plan Adjustment.....	8
4.8 Story 8: Smart Reminders and Notifications.....	9
4.9 Story 9: Personalized Step-by-Step Roadmap Generation.....	10
4.10 Story 10: Goal Analytics and Insights Dashboard.....	11
4.11 Story 11: Multiple Goals Management.....	11
4.12 Story 12: Goal Pause and Resume.....	12
4.13 Story 13: Goal Review and Confirmation Flow.....	14
4.14 Story 14: Goal Status and Health Indicator.....	14
4.15 Story 15: Contextual Learning and Resource Support.....	14
4.16 Story 16: In-App AI Guidance.....	15
4.17 Story 17: Task Completion Notes.....	15
4.18 Story 18: Agentic Workflow Implementation.....	16
5. Story Points by Team Member.....	17
6. Deployment.....	18
7. CI/CD Pipeline.....	18
8. Code Quality.....	18
8.1 Test Summary.....	18
8.2 Coverage.....	19
8.3 Code Smell Analysis (Designite).....	19
8.4 Design Principles.....	19
9. Repository Artifacts.....	20
10. Cumulative Risks and Mitigation Summary.....	21

1. Team Members

Name	Role	Key Contributions
Vijay Puttarevaiah	Full-Stack Developer, DevOps	Goal Pause/Resume, Progress Tracking, CI/CD Pipeline, DPy Integration, Deployment, Code Quality, Documentation
Manavraj Karansinh Thakor	Full-Stack Developer	Goal Creation, Category Detection, Plan Adjustment, AI Chat, Analytics, Multiple Goals, Task Notes
Hinesh Jayeshkumar Patel	Backend Developer, Frontend Auth	User Authentication, Notifications, Roadmap Generation, Auth UI
Aditya Natvarbhai Lad	Backend Developer, Workflows	User Authentication, Agentic Workflows, CI/CD Fixes

2. Project Summary

Metric	Value
Total Story Points	51
Story Points Completed	48
Story Points Remaining	3 (Story 15 - in progress)
User Story Coverage	94%
Total Tests	417 (352 backend + 65 frontend)
Line Coverage	87%
Branch Coverage	85%
Merged Merge Requests	33
Total Commits	255

3. Sprint Overview

Sprint	Duration	Stories Completed
Sprint 1	Jan 26 - Feb 9	System Setup, Architecture, Authentication, Goal Category Detection
Sprint 2	Feb 10 - Feb 23	Goal Creation, Notifications
Sprint 3	Feb 26 - Mar 11	Plan Adjustment, Progress Tracking, Analytics
Sprint 4	Mar 12 - Mar 25	Roadmap, AI Chat, Pause/Resume, Task Notes, Workflows, Multiple Goals
Sprint 5	Mar 26 - Apr 3	Smell Reduction, Deployment, Documentation

4. User Stories - Detailed Breakdown

4.1 Story 1: System Setup and Requirement Gathering

Issue	<u>#1</u>
Story Points	0
Sprint	Sprint 1
Status	Closed
Assigned To	Aditya, Hinesh, Manav, Vijay
Supported Features	Requirement discussions, scope definition, risk identification, and tech stack agreement. No development effort required.

Story Point Rationale

Planning-only story with no technical implementation. All four members participated in scope definition and architecture discussions. 0 SP is appropriate as no code was produced.

Risk Assessment

Unclear requirements could lead to rework in later sprints. Mitigated by documenting scope decisions and maintaining a shared backlog with clear acceptance criteria.

4.2 Story 2: System Architecture

Issue	<u>#2</u>
Story Points	1
Sprint	Sprint 1
Status	Closed
Assigned To	Manav
Supported Features	Layered architecture design (router/service/model separation); React project initialization with Vite; folder structure organization; routing and layout setup; Context API structure for state management; Docker and CI/CD pipeline initial configuration

Story Point Rationale

Although this story included Docker and CI/CD setup, the effort was primarily creating project structure, configuring build tools, and planning the layered architecture. No complex business logic was implemented. The architectural decisions made here (three-layer separation, modular package structure) guided all subsequent stories. 1 SP reflects the planning-heavy, code-light nature of this work.

Risk Assessment

Poor architectural decisions could cascade into technical debt across all future stories. Mitigated by adopting a proven layered architecture (Router → Service → Model) and enforcing separation of concerns from the start.

4.3 Story 3: User Accounts and Secure Access

Issue	<u>#3</u>
Story Points	4
Sprint	Sprint 1
Status	Closed
Assigned To	Aditya (Backend: 2 SP), Hinesh (Frontend: 2 SP)
Supported Features	JWT-based authentication; Register/Login/Logout; Password hashing (bcrypt); Password reset with email verification code; Token blacklisting on logout; Protected routes with authorization middleware; Session persistence via token storage

Key Files

Backend: auth/login/, auth/register/, auth/logout/, auth/email_verification/, auth/password_reset/, auth/utls/auth.py, auth/utls/password.py

Frontend: Login.jsx, Register.jsx, ForgotPassword.jsx, ResetPassword.jsx, SetNewPassword.jsx, VerifyEmail.jsx, VerifyResetCode.jsx, AuthContext.jsx, use_auth.js

Story Point Rationale

This story required implementation across both layers. Backend (Aditya, 2 SP): user registration API, password hashing with bcrypt, JWT token generation, authorization middleware, token blacklisting, password reset logic, and secure data validation. Frontend (Hinesh, 2 SP): 7 page components, form validation, protected routing, AuthContext for session management, and API integration. The security-critical nature added complexity beyond typical CRUD - encryption, token lifecycle, and email verification flows. 4 SP total (2+2) is justified.

Risk Assessment

Security vulnerabilities (SQL injection, token leakage, weak password storage) could compromise user data. Mitigated by using bcrypt for hashing, JWT with expiration, token blacklisting on logout, and centralized authorization middleware that validates every protected request.

4.4 Story 4: Goal Category Detection and Model Selection

Issue	#5
Story Points	4
Sprint	Sprint 1
Status	Closed
Assigned To	Manav (Backend logic: 2 SP), Vijay (Integration and routing: 2 SP)
Supported Features	External service integration for goal text analysis; Prompt engineering for accurate category detection across Career, Fitness, and Immigration domains; Model routing logic based on detected category; Database category storage; Graceful error handling for API failures; Category display in UI without exposing internal system logic

Key Files

Backend: goals/category/service.py, goals/category/router.py, goals/category/prompt_builder.py, clients/llm_client.py, domain/goal_category.py

Frontend: CreateGoal.jsx

Story Point Rationale

This story involved external API integration with significant prompt engineering effort to achieve reliable category detection across multiple domains. The LLMClient class required robust error handling (connection failures, invalid responses, rate limits), JSON response parsing with fallback logic, and health check endpoints. Vijay handled the integration layer - model routing, category enum design, and connecting the detection service to the goal creation flow. The need to test across multiple category types and handle edge cases (ambiguous goals, multi-category overlap) added complexity. 4 SP (2+2 split) is justified.

Risk Assessment

External API unpredictability could return incorrect categories or fail entirely. Mitigated by structured JSON output enforcement, local validation of API responses, fallback error handling in LLMClient, and a health check endpoint (/health/goal_category_llm) to verify provider availability.

4.5 Story 5: Goal Creation and Timeline Definition

Issue	#4
Story Points	6
Sprint	Sprint 2
Status	Closed
Assigned To	Manav
Supported Features	Natural language goal input parsing; Timeline/deadline validation; 30-day chunk batching to keep task generation within token limits; Web research integration via Server API for real-world context; Task generation using external service with research context; Dashboard display with progress bars and task counts

Key Files

Backend: goals/goal/service.py, goals/goal/router.py, goals/ai/llm_service.py, goals/integrations/web_search_service.py

Frontend: CreateGoal.jsx, Dashboard.jsx

Story Point Rationale

This is the highest-complexity story in the project. It involves three external API integrations (category detection, web research, task generation), a chunking algorithm that splits multi-month goals into 30-day batches, web research gathering and injection into generation prompts, JSON parsing with validation, and a full dashboard UI with progress bars. The end-to-end flow (input → categorize → research → generate tasks → store → display) touches every layer of the architecture. 6 SP is justified by the breadth and depth of integration work.

Risk Assessment

External API rate limiting could cause goal creation to fail mid-flow, leaving partially created goals. Mitigated by performing web research once and reusing it across all chunks (avoiding redundant API calls), and by committing the goal to the database before starting task generation so the goal is never lost even if generation fails.

4.6 Story 6: Progress Tracking and Task Completion

Issue	<u>#11</u>
Story Points	3
Sprint	Sprint 4
Status	Closed
Assigned To	Vijay
Supported Features	Task status updates with validation (pending/completed/missed/skipped); Task notes with whitespace sanitization and 1000-char limit; Batch status updates for multiple tasks in a single request; Progress bar calculation based on completed/total ratio; Filter tasks by status (All, Pending, Completed, Missed); Search tasks by title

Key Files

Backend: goals/task/router.py, domain/goal_status.py

Frontend: GoalTask.jsx (TaskCard, FilterPill, NotesModal components)

Tests: test_task_status_update.py, test_task_notes_batch.py

Story Point Rationale

This story required building a complete task interaction layer: status validation against a defined set (VALID_STATUSES), ownership verification before allowing updates, whitespace sanitization on notes, batch update logic that skips unauthorized tasks silently, and a frontend with checkbox toggles, filter pills, and search. The batch update endpoint needed careful handling to commit once for the entire batch rather than per-task. 3 SP is justified by the number of operations and the validation logic required.

Risk Assessment

Task status inconsistencies if multiple users modify simultaneously. Mitigated by using database-level transactions (single db.commit() per batch) and ownership verification via `_get_user_task` helper that raises 403 for unauthorized access.

4.7 Story 7: Adaptive Plan Adjustment

Issue	<u>#12</u>
Story Points	4
Sprint	Sprint 3
Status	Closed
Assigned To	Manav
Supported Features	Missed task detection (pending tasks past due date); Configurable replan threshold (default: 3 missed tasks); Progress-aware task regeneration in 30-day chunks; Safety abort when generation fails (returns 502 without modifying existing plan); Adjustment history logging with task counts and explanation; Trade-off explanation generation; Marks overdue tasks as missed before regenerating

Key Files

Backend: replan/detect/service.py, replan/check/service.py, replan/goal/service.py, replan/routes/router.py

Frontend: GoalTask.jsx (ReplanBanner, ReplanExplanation, AdjustmentHistory components)

Tests: test_detect_missed.py, test_check_replan.py, test_replan_goal.py, test_replan_routes.py, test_replan_llm.py

Story Point Rationale

This is a complex multi-step orchestration: (1) detect missed tasks, (2) build progress summary, (3) gather fresh research, (4) generate replacement tasks in chunks, (5) safety-check the result, (6) mark missed tasks, (7) delete old pending tasks, (8) insert new tasks, (9) generate explanation, (10) log adjustment. The safety-first approach (generate before deleting) adds architectural complexity. The module was decomposed into 4 focused services following SRP. 4 SP is justified.

Risk Assessment

Task generation failure could leave users with no plan if old tasks are deleted prematurely. Mitigated by generating new tasks FIRST before any deletion - if generation returns empty, the function aborts with HTTP 502 and the existing plan remains untouched. This generate-then-swap pattern ensures data safety.

4.8 Story 8: Smart Reminders and Notifications

Issue	<u>#13</u>
Story Points	5
Sprint	Sprint 2
Status	Closed
Assigned To	Hinesh
Supported Features	Background scheduler for daily deadline checks; Overdue task detection and notification creation; Upcoming deadline alerts (configurable time window); Completed task celebration notifications; Email notifications via SMTP with HTML formatting; In-app notification bell with unread count badge; Mark individual notifications as read; Notification preference enforcement (email enabled/disabled per user)

Key Files

Backend: notifications/services/notification_service.py, notifications/services/reminder_service.py, notifications/routers/notifications.py, notifications/models/notification.py

Frontend: Notification bell component in Navbar

Tests: test_notification_service.py (20 tests), test_notification_router.py (12 tests), test_reminder_service.py (24 tests)

Story Point Rationale

This story required: scheduler logic with timezone awareness, deduplication to prevent spam notifications, three different notification types (upcoming, overdue, completed) each with custom message templates, SMTP email integration with HTML body construction, notification preference enforcement, and frontend bell with real-time unread count. The reminder service alone has 24 tests reflecting its complexity. 5 SP is justified by the breadth of backend scheduling logic and the multi-channel notification delivery.

Risk Assessment

Duplicate notifications could spam users if the scheduler runs multiple times. Mitigated by implementing deduplication logic in the reminder service and using database constraints on notification IDs. The DB session is always closed properly (verified by tests `test_db_closed_after_success` and `test_db_closed_on_exception`).

4.9 Story 9: Personalized Step-by-Step Roadmap Generation

Issue	#10
Story Points	4
Sprint	Sprint 4
Status	Closed
Assigned To	Hinesh
Supported Features	Onboarding preference collection (experience level, hours per week, focus areas); Agent-based roadmap generation with 4 specialized agents (coding, fitness, immigration, career); Multi-phase roadmap display with step-by-step progression; Preferences persisted for future access and updates; Agent selection based on goal category detection

Key Files

Backend: `onboarding/services/preference_service.py`, `onboarding/routers/onboarding_router.py`, `goals/roadmap/service.py`, `goals/roadmap/router.py`

Frontend: `Onboarding.jsx`, `Roadmap.jsx`

Tests: `test_preference_service.py` (10), `test_onboarding_router.py` (10), `test_roadmap_service.py` (23), `test_roadmap_api.py` (10)

Story Point Rationale

The roadmap service has 4 distinct agent implementations, each producing domain-specific multi-phase plans. The preference service handles both create and update paths with validation. The onboarding flow required a full UI with step-by-step progression. 53 tests across 4 test files reflect the feature's breadth. 4 SP is justified.

Risk Assessment

Different agent types could produce inconsistent roadmap formats, breaking the frontend display. Mitigated by enforcing a common output structure across all agents and validating the response format before returning to the client.

4.10 Story 10: Goal Analytics and Insights Dashboard

Issue	<u>#14</u>
Story Points	2
Sprint	Sprint 3
Status	Closed
Assigned To	Manav
Supported Features	Progress summarizer with monthly completion stats; Completion rate calculation (handles zero-division safely); Missed task tracking with title display; Compact progress context formatted for downstream consumption; Behind-schedule indicators on dashboard goal cards

Key Files

Backend: goals/progress/summarizer.py

Frontend: Dashboard.jsx (StatCard, BehindBadge, GoalCard components)

Tests: test_progress_summarizer.py (11 tests)

Story Point Rationale

This story is simpler than others because it primarily aggregates existing task data into summary statistics. The main complexity is in `format_summary_for_llm()` which produces a compact text representation that stays under token limits even for multi-year goals (capped at 20 missed tasks in output). The zero-division guard and monthly grouping logic add some edge-case handling. 2 SP reflects the focused, data-aggregation nature of this work compared to the multi-step orchestration in Stories 5 or 7.

Risk Assessment

Large goals (1000+ tasks) could produce progress summaries that exceed token limits for downstream consumers. Mitigated by capping missed task display at 20 items and grouping completion data by month rather than listing individual tasks.

4.11 Story 11: Multiple Goals Management

Issue	<u>#15</u>
Story Points	2
Sprint	Sprint 4
Status	Closed
Assigned To	Manav
Supported Features	Dashboard grid with responsive layout (auto-fill, minmax 260px); Per-goal task counts via database aggregation; Per-goal replan status checks via parallel API calls; Category-based icons and color coding; Paused goal styling (reduced opacity, grey badge); Behind-schedule warning badges

Key Files

Backend: goals/goal/service.py (`get_user_goals`)

Frontend: Dashboard.jsx

Story Point Rationale

This story extends the existing dashboard to handle multiple goals simultaneously. The backend change was minimal (the `get_user_goals` function already existed). The main work was frontend: responsive grid layout, parallel replan status checks via `Promise.allSettled`, and per-card status styling. The replan check exclusion logic for paused/completed goals added some conditional complexity. 2 SP reflects the primarily frontend nature with minimal backend changes.

Risk Assessment

Making N parallel API calls (one per goal) for replan status could be slow with many goals. Mitigated by using `Promise.allSettled` so failures don't block the dashboard, and by skipping replan checks for completed and paused goals.

4.12 Story 12: Goal Pause and Resume

Issue	#16
Story Points	3
Sprint	Sprint 4
Status	Closed
Assigned To	Vijay
Supported Features	Pause active goals (records status and timestamp); Resume with two modes: keep-original-deadline or set-new-end-date; Task regeneration based on completed progress and pause duration; Deadline-passed handling (disables keep-original option); Remaining days warning badge when < 5 days left; All task interactions disabled when paused; Paused goal styling on dashboard; Replan check returns false for paused goals; Replan execution rejects paused goals with 400

Key Files

Backend: `goals/goal/service.py` (`pause_goal`, `resume_goal`, `_get_user_goal`), `goals/goal/schemas.py`, `goals/goal/router.py`

Frontend: `GoalTask.jsx` (`ResumeModal`, pause/resume buttons, paused banner), `Dashboard.jsx`

Tests: `test_pause_resume_goal.py` (27 unit tests), `test_integration_goal_endpoints.py` (23 integration tests)

TDD Commits

Hash	Phase	Description
28ff1a9	RED	Add failing tests for pause/resume
53ba6d7	GREEN	Implement pause/resume with task regeneration
db713e8	GREEN	Add frontend UI with <code>ResumeModal</code>
47cd4a2	REFACTOR	Extract <code>_get_user_goal</code> helper
ba3c888	RED	Add failing tests for resume improvements

3ca7131	GREEN	Skip web research, allow flexible dates
c13921d	GREEN	Add remaining days warning

Story Point Rationale

Full-stack feature requiring: backend service with two distinct resume modes, Pydantic schema with cross-field validation (new_end_date must be future), progress summary integration, task regeneration in 30-day chunks, frontend ResumeModal with radio cards and date picker, conditional UI states (deadline passed, time tight, normal), task interaction disabling, dashboard status updates, and replan integration (skip paused goals). 50 total tests (27 unit + 23 integration) demonstrate the scope. 3 SP is justified.

Risk Assessment

Users resuming after a long pause could get a compressed plan that's unrealistic. Mitigated by passing pause duration and remaining days as context to the task generator, which adjusts task density accordingly. If the generator fails, the function aborts with 502 without modifying the goal (same safety pattern as Story 7).

4.13 Story 13: Goal Review and Confirmation Flow

Issue	<u>#17</u>
Story Points	Not estimated
Status	Open (Not started)
Description	Planned for future development. Not included in current sprint scope.

4.14 Story 14: Goal Status and Health Indicator

Issue	<u>#18</u>
Story Points	Not estimated
Status	Open (Not started)
Description	Planned for future development. Not included in current sprint scope.

4.15 Story 15: Contextual Learning and Resource Support

Issue	<u>#19</u>
Story Points	3
Sprint	Sprint 4

Status	Open (In Progress)
Assigned To	Manav

Current Status: This story is partially implemented. The foundational work for contextual learning was started but not completed within Sprint 4. This accounts for the 3 undelivered story points (51 total - 48 completed = 3 SP remaining).

What Was Started: Initial research and planning for resource integration. The AI Chat feature (Story 16) provides some overlapping functionality by offering goal-level and task-level guidance.

What Remains: Dedicated contextual learning resources, curated content recommendations, and learning path suggestions tied to specific goal categories.

Risk Assessment

Feature scope overlap with AI Chat (Story 16) could lead to redundant functionality. Future implementation should clearly differentiate contextual learning (curated resources) from AI guidance (conversational).

4.16 Story 16: In-App AI Guidance

Issue	<u>#60</u>
Story Points	5
Sprint	Sprint 4
Status	Closed
Assigned To	Manav
Supported Features	Goal-level and task-level chat sessions; SSE streaming responses; Session history and management; Quick task explanations; Suggested follow-up questions; Chat drawer and full-page modes; Rate limiting; Context-aware prompts using goal stats and task details

Key Files

Backend: chat/services/chat_service.py (930+ lines), chat/routers/chat_router.py, chat/models/chat_models.py, chat/schemas/chat_schemas.py

Frontend: ChatPage.jsx, ChatDrawer.jsx, ChatInput.jsx, ChatMessage.jsx, SuggestionChips.jsx, useChat.js, Chatapi.js

Tests: test_chat_service.py (45), test_chat_router.py (15), test_chat_models.py (19), test_chat_schemas.py (17)

Story Point Rationale

This is the second most complex feature after Goal Creation. It required: SSE streaming implementation with chunked response parsing, session management with database persistence, rate limiting logic, context building from goal and task data, system prompt construction, 7 API endpoints, a custom React hook (useChat) for state management, and two distinct UI modes (drawer and full-page). 96 tests across 4 test files. 5 SP is justified by the streaming complexity and the number of components.

Risk Assessment

Streaming responses could hang or produce malformed chunks, breaking the frontend display. Mitigated by implementing `_extract_stream_token()` with JSON parse error handling (logs malformed chunks and continues), and a `[DONE]` sentinel to cleanly terminate streams.

4.17 Story 17: Task Completion Notes

Issue	<u>#51</u>
Story Points	2
Sprint	Sprint 4
Status	Closed
Assigned To	Manav
Supported Features	Add/update task notes with 1000-character limit; Whitespace sanitization; Notes modal with character counter and save confirmation; Batch task status updates in single API call

Key Files

Backend: goals/task/router.py (update_task_notes, batch_update_tasks)

Frontend: GoalTask.jsx (NotesModal component)

Tests: test_task_notes_batch.py (6 tests)

Story Point Rationale

Standard CRUD operations with input sanitization and a modal UI. The batch update endpoint adds some complexity (skip unauthorized tasks, single commit for entire batch). 2 SP reflects the straightforward nature of the work.

Risk Assessment

Users could submit excessively long notes that impact database performance. Mitigated by enforcing a 1000-character limit in the UI and stripping whitespace on the backend before storage.

4.18 Story 18: Agentic Workflow Implementation

Issue	<u>#32</u>
Story Points	3
Sprint	Sprint 4
Status	Closed
Assigned To	Aditya
Supported Features	Goal creation workflow with state management (LangGraph); Replan workflow with research gathering and task generation nodes; Workflow orchestration with defined state transitions; Chunk-based task generation within workflows

Key Files

Backend: workflows/goal_creation_workflow.py, workflows/replan_workflow.py

Tests: test_goal_creation_workflow.py (5), test_replan_workflow.py (5)

Story Point Rationale

This story required designing state machines with multiple nodes (research, generate, validate) and transitions. The LangGraph integration added a learning curve. Each workflow needed to handle intermediate state correctly and support chunk-based iteration for long goals. 3 SP is justified by the state machine complexity and the external framework integration.

Risk Assessment

State machine transitions could leave workflows in inconsistent states if a node fails mid-execution. Mitigated by designing idempotent nodes and validating state at each transition point.

5. Story Points by Team Member

Member	Stories (with SP Split)	Total SP	Additional Contributions
Manav	Architecture (1), Goal Creation (6), Category Detection (2/4), Plan Adjustment (4), Analytics (2), Multiple Goals (2), AI Chat (5), Task Notes (2), Story 15 partial (0/3)	24	Frontend UI for most features
Vijay	Category Detection (2/4), Progress Tracking (3), Goal Pause/Resume (3)	8	CI/CD pipeline (6 stages), AWS EC2 deployment, Docker config, DPy integration, smell reduction (35 files), documentation
Hinesh	User Accounts (2/4), Notifications (5), Roadmap Generation (4)	11	Frontend auth UI (7 pages), notification preferences
Aditya	User Accounts (2/4), Agentic Workflows (3)	5	CI/CD fixes, backend auth implementation
Total		48	

Note: While story point counts vary across team members, contributions beyond story-pointed work are significant. Vijay's DevOps responsibilities (CI/CD pipeline with 6 stages, AWS EC2 deployment, Docker configuration, DPy integration, smell reduction across 35 files, and project documentation) represent substantial effort not captured in story points. Similarly, Aditya's CI/CD troubleshooting and Hinesh's 7-page authentication UI flow contributed meaningfully beyond their SP-counted stories.

6. Deployment

Environment	Frontend	Backend API	API Docs
Development	http://18.117.130.226:8081	http://18.117.130.226:8073	http://18.117.130.226:8073/docs
Production	http://18.117.130.226:8082	http://18.117.130.226:8074	http://18.117.130.226:8074/docs

Infrastructure: AWS EC2 t3.micro (us-east-2), Docker containers, MySQL 8.0, Nginx reverse proxy for frontend

7. CI/CD Pipeline

Stage	Jobs	Trigger
build	backend-build, frontend-build	All branches (on file change)
test	backend-test, backend-test-coverage, frontend-test	All branches (on file change)
run-dpy	run-dpy-job (Designite code smell analysis)	All branches (backend changes only)
submit-dcode	submit-dcode-job (DCode quality submission)	All branches (backend changes only)
publish	publish (Docker image build and push to Docker Hub)	main and develop branches only
deploy	deploy (SSH to EC2, pull images, restart containers)	main and develop branches only

8. Code Quality

8.1 Test Summary

Category	Count
Backend Unit Tests	329
Backend Integration Tests	23
Frontend Tests	65
Total	417

8.2 Coverage

Metric	Value
Line Coverage	87%
Branch Coverage	85%

8.3 Code Smell Analysis (Designite)

Category	Detected	Status
Architecture Smells	0	Clean (Feature Concentration was resolved)
Design Smells	46	All justified with Status and Justification columns
Implementation Smells	196	Reduced from 280; mostly false positives (Long Statement from f-string expansion)

Reports with justification columns are stored in docs/smells/.

8.4 Design Principles

Principle	Application
Single Responsibility	Each module has one purpose (e.g., detect/service.py only detects, check/service.py only checks)
Open/Closed	New categories added to domain/goal_category.py without modifying services
Dependency Inversion	Services depend on abstract get_db, overridden in tests with in-memory SQLite
DRY	Shared constants (domain/goal_status.py), shared frontend utils (utils/designTokens.js), shared JSON parsing (goals/ai/llm_service.py)
High Cohesion	core/ has LCOM near 0 (only DB + logging). Feature Concentration resolved by moving password.py to auth/utils/ and llm_client.py to clients/
Low Coupling	One-directional dependencies: Router → Service → Model. No circular imports.

9. Repository Artifacts

Artifact	Location
Root README	README.md
Final Report	docs/Final_Deliverables_Report.md
Architecture Smells	docs/smells/ArchitectureSmells.csv
Design Smells	docs/smells/backend_design_smells.csv
Implementation Smells	docs/smells/backend_implementation_smells.csv
Class/Module Metrics	docs/smells/backend_class_module_metrics.csv
Function Metrics	docs/smells/backend_function_metrics.csv
CI/CD Pipeline	.gitlab-ci.yml
Docker Configuration	docker-compose.yml

10. Cumulative Risks and Mitigation Summary

In addition to the per-story risk assessments provided in Section 4, the following table provides a consolidated view of cross-cutting risks and their mitigation strategies.

Risk	Affected Stories	Mitigation
External API rate limiting	Stories 4, 5, 7, 12	Retry mechanism; resume skips web research; single research pass reused across chunks
Task generation failure	Stories 5, 7, 12	Safety abort: generate tasks FIRST, only delete old tasks after successful generation. Returns 502 without modifying existing plan.
Security vulnerabilities	Story 3	JWT with expiration; bcrypt password hashing; token blacklisting; centralized authorization middleware
Notification spam	Story 8	Deduplication logic; database constraints; proper DB session cleanup
Stale tasks after long pause	Story 12	Resume regenerates tasks based on actual completed progress and pause duration context
Streaming response failures	Story 16	JSON parse error handling in stream parser; [DONE] sentinel for clean termination
EC2 IP changes on restart	Deployment	Pipeline and CI/CD variables updated; Elastic IP recommended for production

Docker Hub rate limits	CI/CD	Retry failed jobs; rate limits are per-IP on shared university runners
-------------------------------	-------	--